بسم الله الرحمن الرحیم

Data and Network Security

SESSION 03

Access Control Models and Data Protection



Solmaz Salimi
Sharif University of Technology

s.salimi@sharif.edu

Last lecture and today's lecture

Session 2	Enforcement points, delegated authority, and current policy state
Part 1	DAC, access matrix, ACLs, and capabilities
Part 2	MAC, Bell–LaPadula, and Biba
Part 3	RBAC, sessions, hierarchy, and separation of duty
Part 4	Information-flow control and noninterference
Review	One export combines identity, object, label, and output checks

The export service needs an explicit policy

SUBJECTS

Alice: approved
Bob: no approval
Worker and steward



OBJECTS

Dataset A
Export archive
Audit record



RULES

Object grants
Project roles
Secrecy and integrity labels

The models express different policy dimensions; they are not interchangeable.

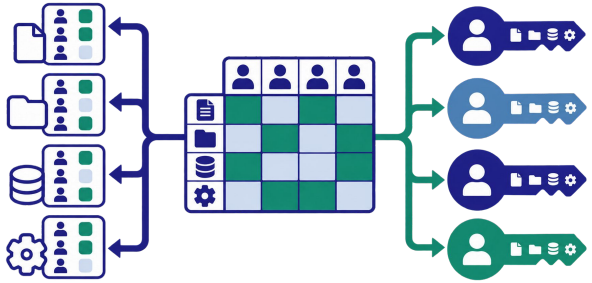
An authorization decision evaluates a structured request

Subject	Object	Operation	Context	Policy state
User or process	Dataset A	read	Project and time	Grants and labels

allow = Policy(subject, object, operation, context, state)



Part 1: Discretionary access control

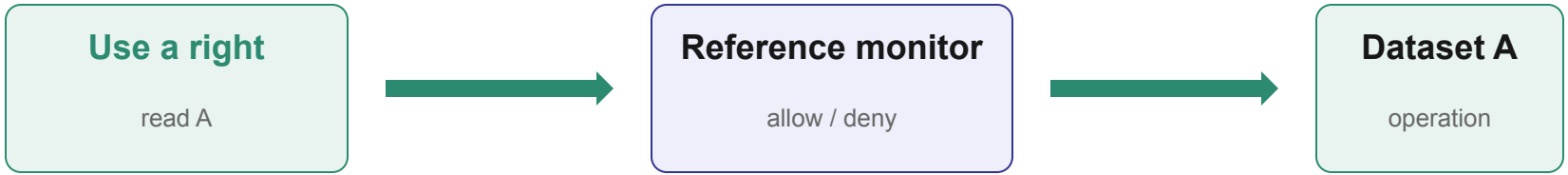


An access matrix records the current rights

Subject	Dataset A	Archive	Grant policy	Audit log
Alice	read	none	request	none
Bob	none	none	request	none
Worker	read	write	none	append
Steward	none	approve	edit	read

At this policy snapshot, Alice is approved for A and Bob is not. Each cell is a set of rights for one subject–object pair.

Policy administration changes the protection state



grant
add a right

revoke
remove a right

create
add subject/object

delete
remove subject/object

DAC gives authorized users discretion over object rights. Grant and revoke operations change the protection state and require their own authority.

ACLs and capability lists represent different matrix views

Object-centered

ACL = one matrix column

- Stored with the object
- Lists subjects and rights
- Review grants for one object

Subject-centered

Capability list = one matrix row

- Held by the subject
- Names objects and rights
- Delegate authority to one subject

An ACL (Access Control List) stores rights by object. A capability list stores the authority held by a subject.

An access-control list identifies an object’s permitted users



Dataset A ACL

Researcher Alice	read
Approved Project A	read
Export worker	read
Data steward	manage grants
Others	none

Effective access

Effective access follows the ACL’s rules for direct, group, and inherited grants.

UNIX mode bits select one permission class

```
-rw-r-----  1  alice  research  dataset-a
```

OWNER	rw-	Alice can read and write
GROUP	r--	research group members can read
OTHER	---	everyone else has no access

Permission class

Match owner first, then group, otherwise other. Use only that class.



passwd and the setuid Bit



Assume /usr/bin/passwd is owned by root and installed with mode 4755:
Alice, whose UID is 1000, executes passwd.

```
-rwsr-xr-x  root root  /usr/bin/passwd
```

Process	Real UID	Effective UID
Alice's shell	1000	1000
passwd after execution	1000	0

The setuid bit changes the executable's effective UID to its owner's UID. The real UID remains Alice's.

For local accounts, privileged password-update code can modify protected data in /etc/shadow. The password-change policy permits an ordinary user to change only their own password and applies the configured authentication checks.

The kernel provides the program's authority. The program must enforce which operation the caller may request.

An Incorrect Authorization Check

Hypothetical setuid helper

```
if (geteuid() == 0) {
    change_password(target_user);
}
```

Root-owned; executing with elevated authority.

Alice requests Bob's password change

Alice's real UID is 1000.
The helper's effective UID is 0.
The check succeeds because the helper has root authority.

If no further authorization check exists, Alice can change Bob's password.

The check tests the helper's privilege rather than the caller's permission to modify the target account.

Correct enforcement identifies the invoking account using its real UID, checks authorization for the target account, and applies the required authentication policy.

This is an illustrative implementation error, not a claim about the current passwd implementation.

Other Vulnerabilities in setuid Programs

Implementation error	Possible consequence
Check a pathname with access(), then open it separately	The caller changes the pathname between checking and opening. Privileged code accesses a different object.
Locate a helper through a caller-controlled PATH while retaining privilege	An attacker-selected executable may run with the program's elevated authority.
Ignore failure when dropping privileges with setuid()	Execution continues with privileges the program assumed it had dropped.

A setuid program must control both the requested operation and the execution context in which it uses its authority.

A process can inherit the user's ambient authority

User Alice

may read Dataset A

→

Downloaded tool

same effective UID

→

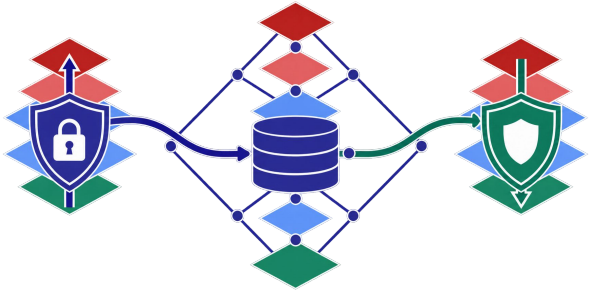
Copy of A

file readable by others

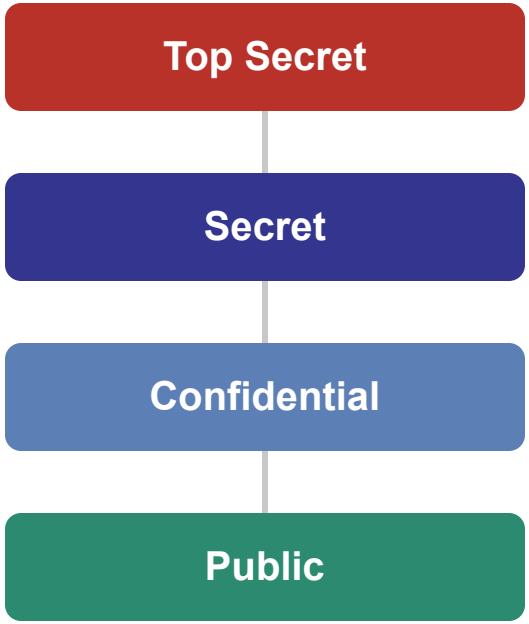


With ordinary UNIX DAC, a process running under Alice's identity can read A and create another copy using her file permissions.

Part 2: Mandatory access control



MAC uses a label lattice and a dominance relation



Label = level + categories

Label = (level, category set)

$L_1 \geq L_2$ iff $level_1 \geq level_2$ and $categories_1 \supseteq categories_2$.

Mandatory policy assigns labels and access rules; ordinary owners cannot override them.

Bell–LaPadula defines permitted confidentiality flows



Simple-security property

NO READ UP

Read o only if L(o) ≤ L(p).

*-property

NO WRITE DOWN

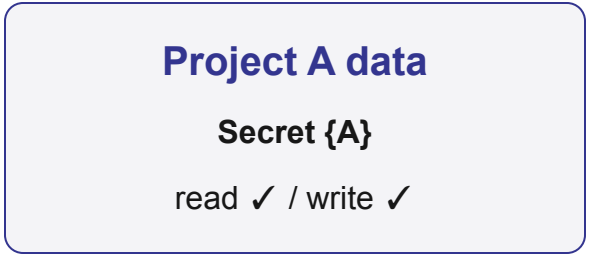
Write o only if L(p) ≤ L(o).

Here L(p) is the process's fixed current label, bounded by its clearance. A process cannot lower its label after reading secret data.

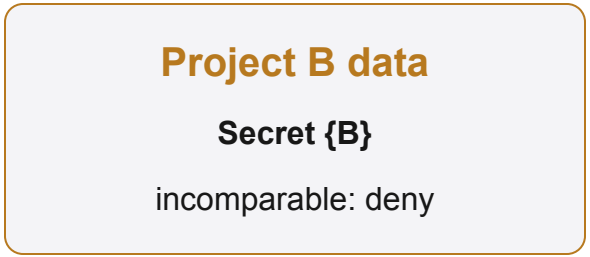
Applying the Bell–LaPadula rules











Use the process's current label, Secret {A}. These label checks are additional to any required discretionary permission.

MAC Example: A Restricted Report Inbox

A reporting process runs with the fixed label Secret {A}. It reads Project A data and submits a report to an inbox labeled Top Secret {A}.
Under Bell–LaPadula, the label checks give these results:

Operation	Result
Read data labeled Secret {A}	**Allowed**
Append a report to Top Secret {A}	**Allowed**
Read messages in Top Secret {A}	**Denied**
Write the report to Public	**Denied**
Write the report to Top Secret {B}	**Denied**

Writing up places information in a more restrictive destination. It does not grant permission to read that destination.

The append operation returns no existing message contents. Object permissions must also allow the operation.

Strict Biba defines permitted integrity flows



Simple integrity property

NO READ DOWN

Read o only if I(p) ≤ I(o).
A process reads data of equal or higher integrity.

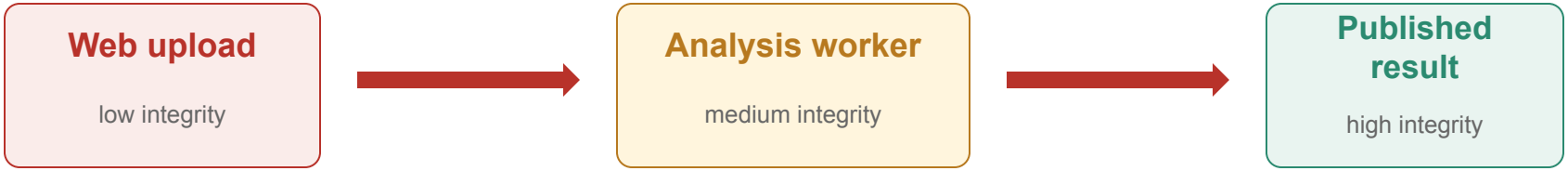
Integrity *-property

NO WRITE UP

Write o only if I(o) ≤ I(p).
A process writes data of equal or lower integrity.

The integrity order describes trustworthiness of data, not its secrecy classification.

Strict integrity labels constrain a processing pipeline



Apply strict Biba to each operation

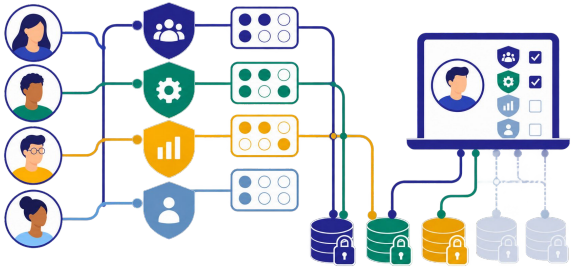
A medium-integrity worker cannot read the low-integrity upload or write the high-integrity published result.

A trusted endorsement procedure validates specified properties before assigning a higher integrity label.

Bell–LaPadula and Biba are mirror policies

Question	Bell–LaPadula	Strict Biba
Primary goal	Confidentiality	Integrity
Read rule	No read up	No read down
Write rule	No write down	No write up
Prevent	Secret data leaking lower	Untrusted data contaminating higher
Trusted exception	Declassification	Authorized endorsement
Does not solve	Integrity, availability, covert channels	Confidentiality, availability, covert channels

Part 3: Role-based access control



Core RBAC separates user assignment from permission assignment

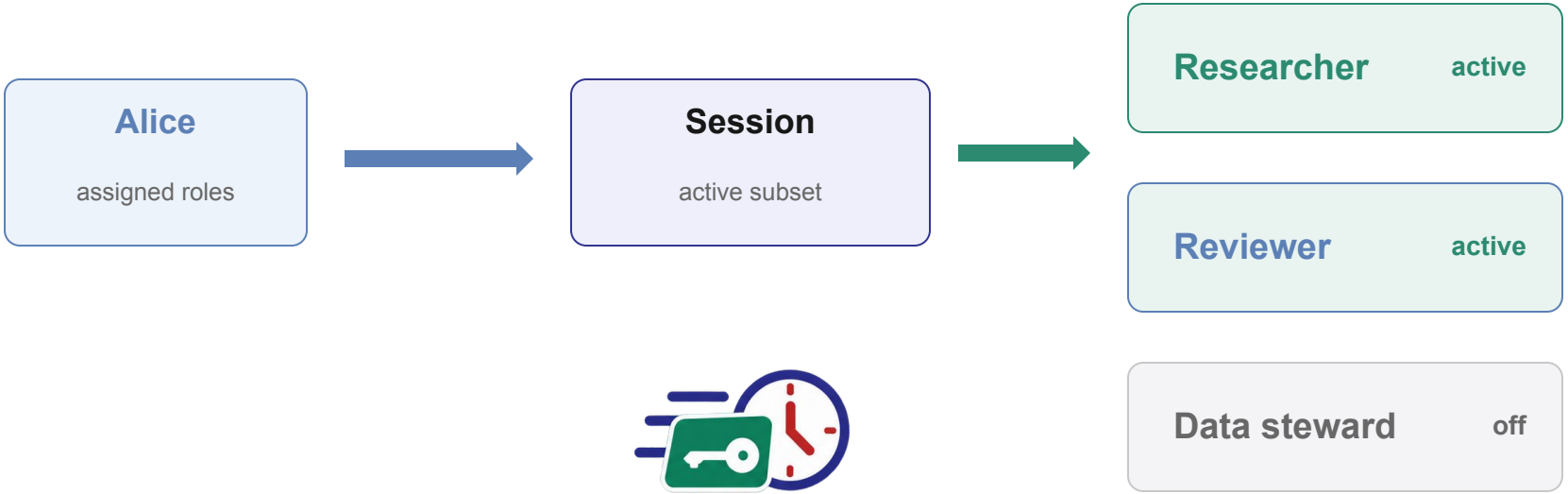


UA: user-role assignment

PA: permission-role assignment

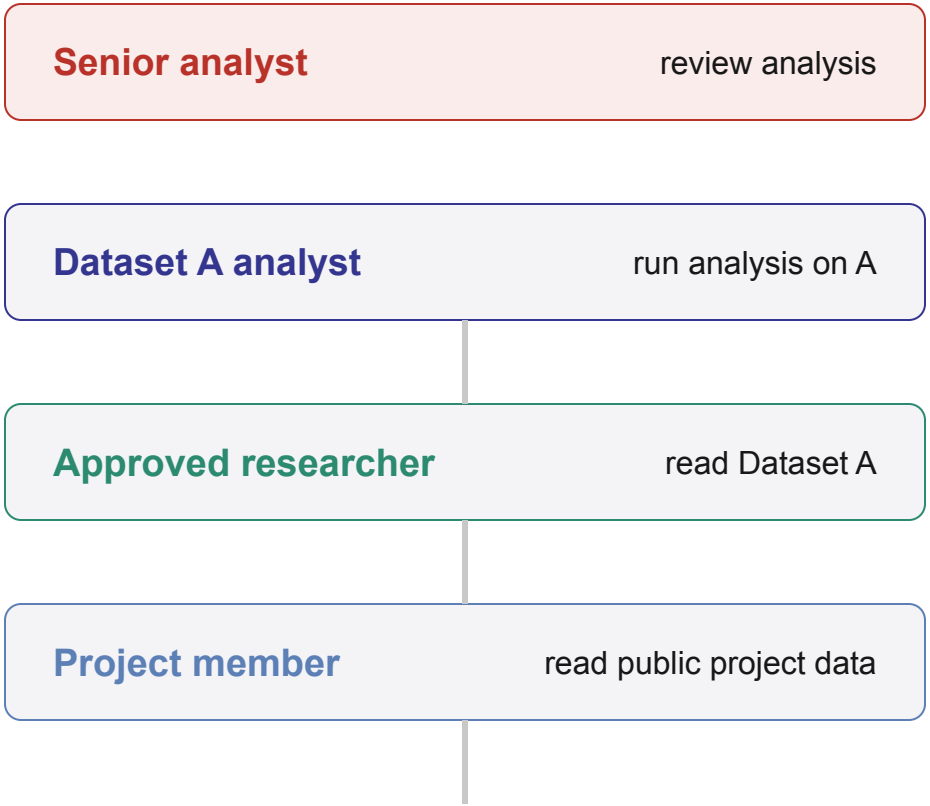
A request is permitted when an active authorized role provides the required operation–object permission.

A session activates only the roles needed now



A session activates a subset of Alice's assigned roles. Inactive roles do not contribute permissions to that session.

Role hierarchies reuse permission bundles



The senior role includes the junior role's permissions.

Export approval is a separate role.

Separation of duty constraints role combinations



Static separation

A user may not be assigned both Requester and Approver.

Dynamic separation

Both roles may be assigned, but not active in the same session.

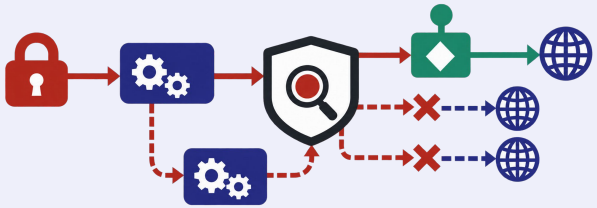
Dynamic separation of duty constrains a session. Transaction-specific separation also needs the request's history.

Request history enforces separation across sessions

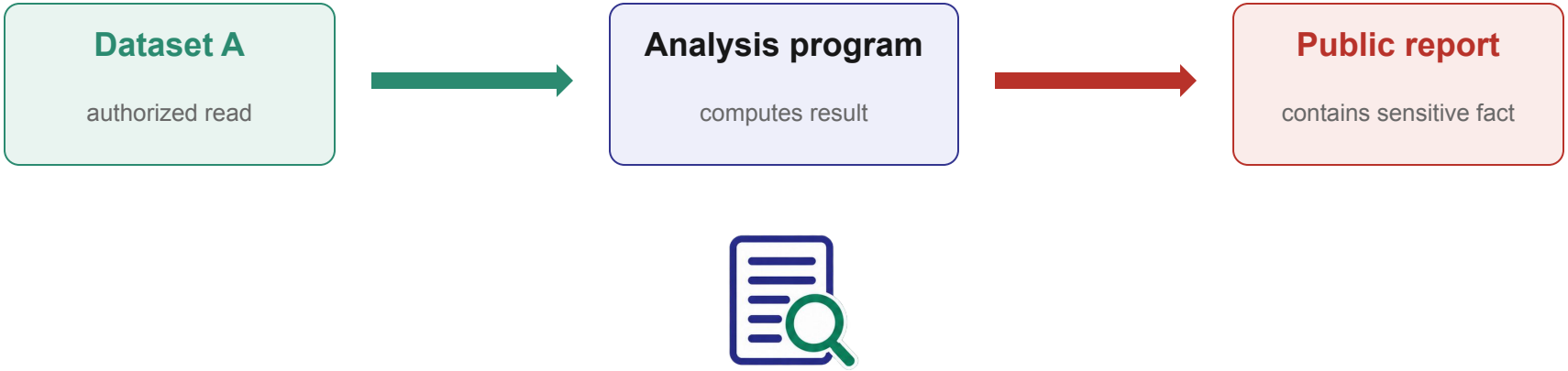
Session α	Alice activates Requester and submits export request R.
Session β	Alice activates Approver. Session separation still holds.
Decision	For request R, require requester_id ≠ approver_id.
Record	Store both identities and enforce the rule at approval.

At approval, compare the stored requester with the approver of the same R. Changing sessions does not change Alice's identity.

Part 4: Information-flow control



An authorized read creates a second policy question



The program may read A. A confidentiality policy must also specify whether a public result may depend on A’s contents.

Data and control dependencies both carry information

Explicit flow

```
public = secret
```

The assignment copies the secret value.

Implicit flow

```
if (secret)
  public = 1
else public = 0
```

The branch writes a value determined by the secret.

Noninterference defines independence of public observations

Program	Let P be deterministic and terminate on all allowed inputs. Let l be public input and h secret input.
Condition	For every l, h_1, h_2: $low(P(l, h_1)) = low(P(l, h_2))$.
Observer	The function low selects the outputs visible to the public observer.
Example	The previous branch produces 0 or 1 as h changes, so its public output is not independent of h.
Scope	Timing, errors, and other channels belong to the property only when included in the observation model.

Enforcing information-flow rules

Static security types

Security types constrain assignments and branches.

Soundness depends on the modeled language.

Dynamic taint tracking

Propagate labels through executed operations.

A program-counter label tracks control context.

OS or process labels

A label applies to a whole process or channel.

This is coarser than per-value tracking.

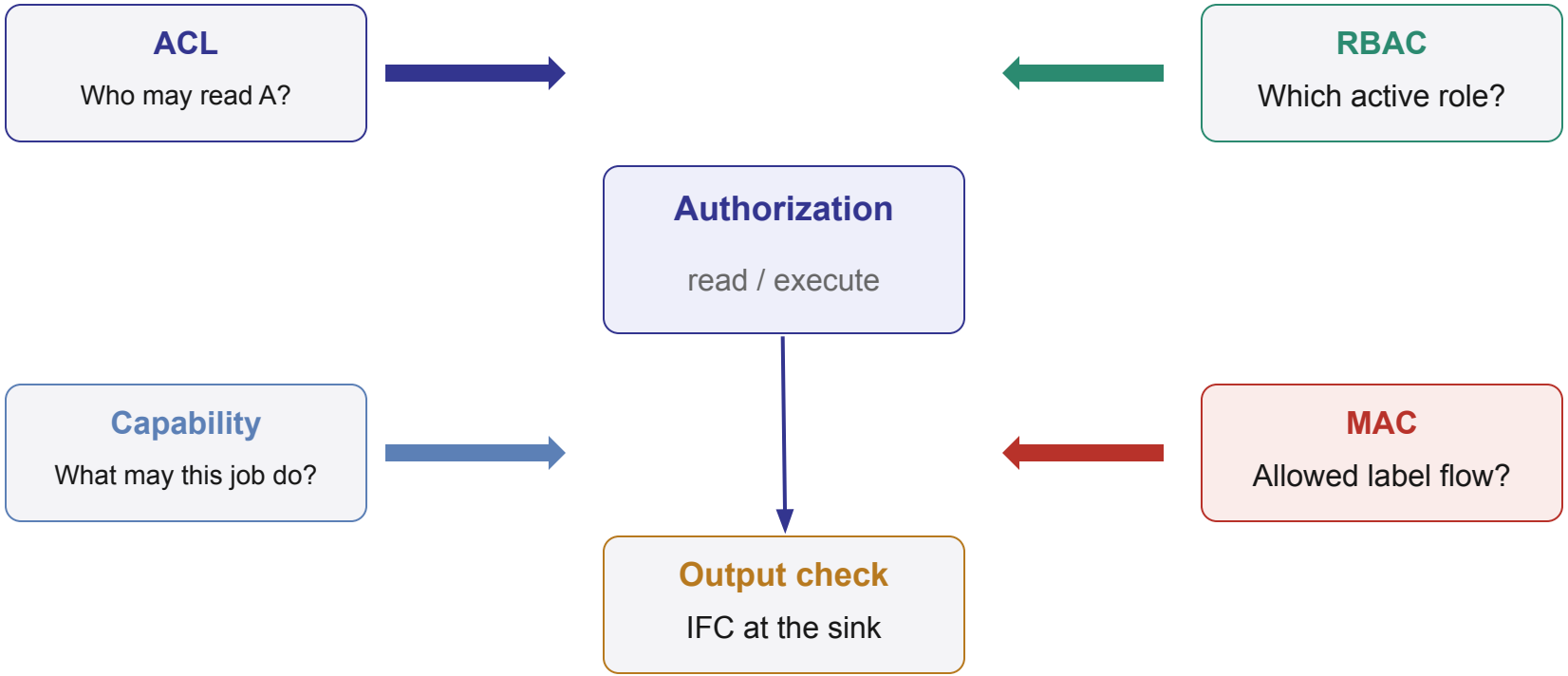
Hybrid enforcement

Combine static and runtime enforcement.

Specify what each mechanism mediates.

Declassification authorizes selected secrecy releases; endorsement authorizes selected integrity upgrades.

Combining policy models in an export service



An export must satisfy all required object, role, delegation, and output rules.

Worked export: access to A and release of a result

Alice requests export R of Dataset A.	Her active role permits requesting exports.
Check Alice’s current read grant for A.	A role alone does not select the dataset.
Delegate the read to the worker for R.	The capability covers A, read, and R.
The process reads at label Secret {A}.	Both discretionary and label rules permit it.
The ordinary result targets Public.	A secret-dependent write is not permitted.
A trusted release procedure selects content.	Its policy authorizes a particular declassification.

Authorization permits reading A for R. Declassification separately defines which information the public result may reveal.

References

DAC	Trent Jaeger, Operating Systems Security, Chapter 4: Security in Ordinary Operating Systems.
RBAC	NIST RBAC model and FAQ: role assignment, sessions, hierarchies, and separation of duty.
IFC	Sabelfeld and Myers, Language-Based Information-Flow Security, IEEE JSAC, 2003.

MAC: Bell–LaPadula and strict Biba. Cornell CS 5430 lecture notes.

Policy structure determines the enforcement task

DAC	Owners and authorized subjects delegate object rights.
MAC	Mandatory labels govern permitted accesses and flows.
RBAC	Active roles supply operation–object permissions.
IFC	Public observations are constrained by data dependencies.

Next: C memory and call stacks. We examine the implementation beneath these enforcement mechanisms.